

SDEV2171

Lesson 15: Global State Concepts

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- explain the difference between local screen state and shared state
- identify a multi-screen scenario where copied data becomes fragile
- recognize the limitations of passing mutable data through navigation parameters
- analyze where one shared source of truth is needed
- explain why Context is the next lesson instead of jumping to more parameter passing

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson Roadmap

1. Reconnect lesson 14 to a new multi-screen problem
2. Define local state, duplicated state, and shared state
3. Inspect one copied-parameter workflow
4. Name the mismatch and its cause
5. Sort values into local versus shared categories
6. Checkpoint and lesson-16 handoff

WE ARE ESSENTIAL TO ALBERTA | NAIT

Starting Point From Lesson 14

- keep the same local backend and `{ id, label }` study-step data
- keep the same multi-screen Expo Router flow
- keep the same request success path

What changes today

- one screen can build a selected study list
- another screen sees only a copied version of that list
- the two screens can drift apart

WE ARE ESSENTIAL TO ALBERTA | NAIT

Local State Still Has A Job

- local state is still correct for values that belong to one screen only
- examples:
  - loading indicators
  - temporary input text
  - one open or closed UI section

Important rule

- local state is not wrong
- it only becomes wrong when multiple places need one changing value

WE ARE ESSENTIAL TO ALBERTA | NAIT

Three State Situations Today

1. local state
2. duplicated state
3. shared state

Quick definitions

- `local state` : one screen owns it
- `duplicated state` : more than one screen keeps its own copy
- `shared state` : one source of truth serves more than one screen

WE ARE ESSENTIAL TO ALBERTA | NAIT

What Navigation Parameters Do Well

- open another screen
- carry one fixed identifier
- carry one short snapshot or label

What they do poorly

- keep two changing screens synchronized
- act as the main home for mutable shared data

WE ARE ESSENTIAL TO ALBERTA | NAIT

Problem Demo Setup

- details screen fetches study steps
- details screen tracks selected steps with local state
- plan screen receives a copied parameter version of that selection

Question to watch

- What happens when one screen changes its copy after navigation?

WE ARE ESSENTIAL TO ALBERTA | NAIT

Clue To Reproduce

1. add two study steps on the details screen
2. open the plan copy screen
3. remove one item there
4. go back to the details screen

Expected clue

- the details screen still shows the old selection count

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why The Screens Drift

- the plan screen did not get the same state
- it got a copied value through navigation
- after that copy exists, each screen can change independently

Plain-language version

- two screens now believe different things about the same list

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Real Question

- not “Can I force this to work with more parameters?”
- but “Where should this data actually live?”

Better framing

- which values are private to one screen
- which values represent one shared app feature

WE ARE ESSENTIAL TO ALBERTA | NAIT

Sorting State Values

Keep local

- loading or error UI for one request
- temporary draft text before submit
- one open or closed card

Probably shared later

- selected study plan used across screens
- favorites used by more than one route
- one user profile shown and edited in multiple places

WE ARE ESSENTIAL TO ALBERTA | NAIT

Common Wrong Turns

- passing larger and larger objects through route params
- copying the same value into two screens and hoping they stay aligned
- treating every state value as global
- jumping to Context before naming the real problem clearly

WE ARE ESSENTIAL TO ALBERTA | NAIT

Midpoint Check

By the middle of class, students should know:

- which value is duplicated
- which two screens drift apart
- what real clue proves the mismatch
- whether the value should stay local or become shared

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

Students should be ready to explain

- one value that should stay local
- one value that should become shared
- one reason copied parameter data is fragile here
- one clue that proves the screens are out of sync

Status language

- `resolved` : you can explain the mismatch and the needed shared-state direction clearly
- `partial` : you can name the screens, but the mismatch or state decision is still fuzzy
- `blocked` : you cannot yet reproduce or explain the drift

WE ARE ESSENTIAL TO ALBERTA | NAIT

What We Learned Today

- local state is still useful, but it is not enough for every feature
- copied mutable data can drift out of sync across screens
- route parameters are good for navigation context, not for long-term shared mutable state
- the next step is one shared source of truth

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Context Is Next

- Context is a React tool for sharing one value across multiple screens or components
- it helps multiple places read the same source of truth instead of keeping copied versions
- it does not replace every local state value

What lesson 16 will do

- create one shared home for the cross-screen value
- let both screens read and update that same value

WE ARE ESSENTIAL TO ALBERTA | NAIT

Next Step

- keep the problem example small and clear
- do not add more copied state branches
- prepare to create one shared source of truth with Context in lesson 16

Ready after lesson 15 means

- `resolved` : you can name the state-sharing problem precisely
- `partial` : you need one more comparison or clue before the problem is clear
- `blocked` : you need help reproducing or interpreting the mismatch

WE ARE ESSENTIAL TO ALBERTA | NAIT